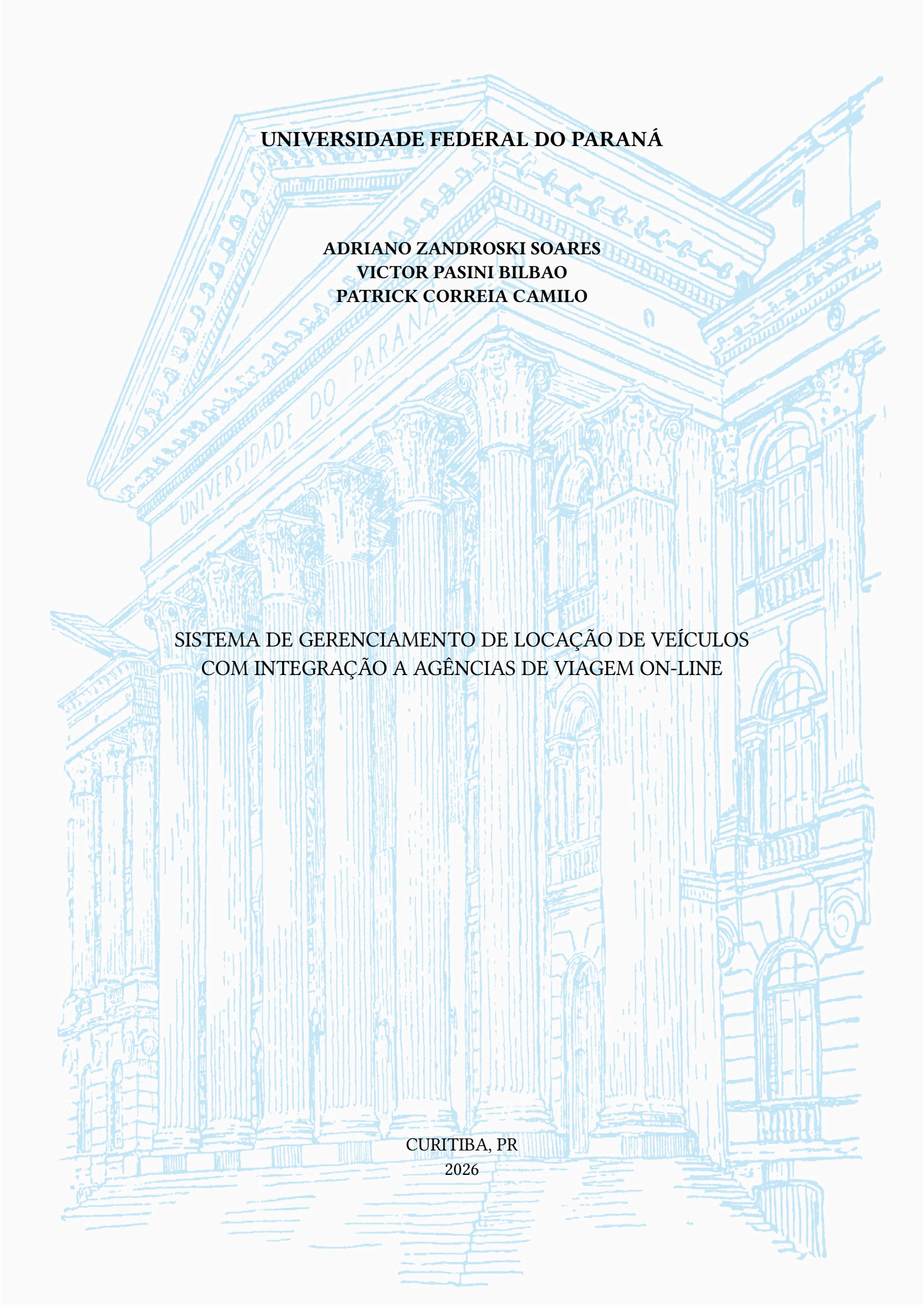




UNIVERSIDADE FEDERAL DO PARANÁ

**ADRIANO ZANDROSKI SOARES
VICTOR PASINI BILBAO
PATRICK CORREIA CAMILO**

**SISTEMA DE GERENCIAMENTO DE LOCAÇÃO DE VEÍCULOS
COM INTEGRAÇÃO A AGÊNCIAS DE VIAGEM ON-LINE**

**CURITIBA, PR
2026**

**ADRIANO ZANDROSKI SOARES
VICTOR PASINI BILBAO
PATRICK CORREIA CAMILO**

**SISTEMA DE GERENCIAMENTO DE LOCAÇÃO DE VEÍCULOS
COM INTEGRAÇÃO A AGÊNCIAS DE VIAGEM ON-LINE**

Trabalho de conclusão de curso apresentado ao curso de
graduação em Análise e Desenvolvimento de Sistemas, Setor de
Educação Profissional e Tecnológica, Universidade Federal do
Paraná, como requisito parcial à obtenção do título de tecnólogo.

Orientador: Prof. Alexander Robert Kutzke.

CURITIBA, PR
2026

Insira aqui o termo de aprovação do TCC, fornecido após a aprovação em banca.

CarRental Marketplace: uma Plataforma B2B2C para Locação de Veículos com Arquitetura Multi-Filial e Integração com Parceiros OTA

Adriano Zandroski Soares
Setor de Educação Profissional e
Tecnológica
Universidade Federal do Paraná
Curitiba, Paraná, Brasil

Victor Pasini Bilbao
Setor de Educação Profissional e
Tecnológica
Universidade Federal do Paraná
Curitiba, Paraná, Brasil

Patrick Correia Camilo
Setor de Educação Profissional e
Tecnológica
Universidade Federal do Paraná
Curitiba, Paraná, Brasil

Abstract—O mercado brasileiro de locação de veículos é majoritariamente atendido por sistemas de gestão fechados, voltados a uma única locadora, sem suporte nativo à operação multi-filial, à composição de um *marketplace* que reúna diferentes locadoras sob um único ponto de busca, ou à integração com agências de viagem *on-line* (OTAs). Este artigo apresenta o desenvolvimento do CarRental Marketplace, uma plataforma B2B2C (*business-to-business-to-consumer*) de locação de veículos que conecta locadoras, suas filiais e clientes finais em um único sistema web, com pagamento no destino (*pay at desk*). A plataforma foi construída sobre uma arquitetura cliente-servidor desacoplada — *backend* em Python com FastAPI, persistência em SurrealDB (banco multimodelo de documentos e grafos) e *frontend* em SvelteKit — e contempla autenticação por papéis via JSON Web Tokens, um motor de precificação, um controle de disponibilidade para evitar *overbooking*, uma máquina de estados para o ciclo de vida das reservas e uma API dedicada para parceiros OTA. O desenvolvimento foi guiado pelo levantamento de 49 histórias de usuário e documentado por diagramas de classes de domínio e de sistema, pelo modelo lógico do banco de dados e por diagramas de sequência para cada tela. Como resultado, obteve-se uma plataforma funcional que materializa, em código, os conceitos de domínio levantados na análise.

Index Terms — locação de veículos; *marketplace* B2B2C; SaaS multi-filial; FastAPI; SurrealDB; SvelteKit; integração OTA.

Abstract — The Brazilian vehicle rental market is largely served by closed, single-tenant management systems designed for a single rental company, with no native support for multi-branch operations, for assembling a marketplace that brings together different rental companies under a single search point, or for integration with online travel agencies (OTAs). This article presents the development of the CarRental Marketplace, a B2B2C (*business-to-business-to-consumer*) vehicle rental platform that connects rental companies, their branches, and end customers within a single web system, following a pay-at-desk model. The platform was built on a decoupled client-server architecture — a Python/FastAPI backend, persistence in SurrealDB (a multi-model document-and-graph database), and a SvelteKit frontend — and implements role-based authentication via JSON Web Tokens, a pricing engine, an availability ledger to prevent overbooking, a state machine governing the reservation lifecycle, and a dedicated API for OTA partners. Development was guided by the elicitation of 49 user stories and documented through domain and system class diagrams, the logical data model,

and sequence diagrams for each screen. As a result, a functional platform was produced that materializes, in code, the domain concepts identified during analysis.

Index Terms: vehicle rental; B2B2C marketplace; multi-branch SaaS; FastAPI; SurrealDB; SvelteKit; OTA integration.

I. INTRODUÇÃO

O setor de locação de veículos é um elo central da cadeia de mobilidade e turismo. No Brasil, esse mercado é numericamente dominado por empresas de pequeno e médio porte: em 2025, o país chegou a 37.047 locadoras em atividade, um crescimento de 17,7% em relação ao ano anterior, faturando R\$ 61,7 bilhões e administrando uma frota recorde de 1.717.848 veículos [1], [2]. Boa parte dessas empresas opera uma ou poucas filiais e não possui capacidade técnica ou orçamentária para desenvolver — ou mesmo contratar — uma plataforma própria de reservas com o mesmo nível de integração das grandes redes internacionais, que mantêm sistemas próprios conectados a agências de viagem *on-line* (OTAs).

Essa lacuna é normalmente preenchida por dois extremos pouco satisfatórios: de um lado, controles manuais que não escalam para múltiplas filiais nem evitam o *overbooking* — a venda do mesmo veículo, ou da mesma categoria, para mais de um cliente no mesmo período; de outro, sistemas de gestão fechados, voltados a uma única empresa, sem suporte nativo à operação de um *marketplace* que reúna várias locadoras concorrentes sob um único ponto de busca para o cliente final, nem à exposição de uma API para que parceiros OTA comercializem as diárias dessas locadoras.

Esse cenário se conecta a uma tendência mais ampla de intermediação digital entre empresas e consumidores finais, estudada na literatura de economia de plataformas como mercados de dois ou múltiplos lados (*two-sided markets*), em que uma plataforma central coordena a oferta de diferentes empresas e a demanda dos consumidores, capturando valor da rede gerada por essa intermediação [3]. No mercado, esse tipo de arranjo é popularmente referido pelo termo B2B2C (*business-to-business-to-consumer*).

O objetivo deste trabalho é apresentar o desenvolvimento do **CarRental Marketplace**, uma plataforma B2B2C de locação de veículos que conecta locadoras, filiais e clientes

finais em um único sistema web, com pagamento no destino. O restante deste artigo está organizado da seguinte forma: a Seção II discute os trabalhos e fundamentos teóricos e tecnológicos que sustentam as decisões de projeto; a Seção III descreve a arquitetura do sistema; a Seção IV apresenta a modelagem de domínio, de sistema e de dados; a Seção V detalha as principais funcionalidades implementadas; a Seção VI resume a especificação das interfaces de programação (APIs); a Seção VII discute os resultados e limitações identificadas; e a Seção VIII apresenta a conclusão.

II. FUNDAMENTAÇÃO TEÓRICA E TECNOLÓGICA

A operação de uma locadora de veículos é, em essência, um problema de gestão de um inventário perecível e distribuído: cada veículo, em cada filial, representa uma unidade de capacidade que só pode ser vendida a um cliente por vez e que se perde de forma irreversível caso a diária não seja comercializada naquele intervalo de tempo. Esse problema se agrava quando a locadora opera mais de uma filial, pois a disponibilidade de cada categoria de veículo precisa ser controlada filial a filial, e quando se permite a devolução em uma filial diferente da retirada (*one-way*), o que exige transferir a contagem de frota disponível entre as duas unidades e cobrar uma taxa adicional pelo deslocamento do veículo. Soma-se a isso a precificação, que raramente é um valor único por categoria: ela costuma variar por plano tarifário (com regras de elegibilidade por antecedência de reserva, idade do condutor ou código promocional), por taxas específicas de cada filial e por adicionais e proteções opcionais escolhidos pelo cliente no momento da reserva.

Atualmente, esse problema é resolvido majoritariamente por sistemas de gestão de locação fechados, contratados ou desenvolvidos por uma única locadora para controlar exclusivamente a própria frota e as próprias filiais. Esses sistemas concentram seus esforços na operação interna — cadastro de veículos, contratos e clientes — e, com frequência, tratam a tarifação de forma simplificada, variando o preço apenas por parâmetros básicos como data de retirada, data de devolução e, eventualmente, idade ou nacionalidade do condutor, sem modelar nativamente taxas decorrentes de decisões operacionais, como a taxa de devolução *one-way* entre filiais, que muitas vezes acaba sendo cobrada manualmente, fora do sistema. Além disso, por serem desenhados para uma única empresa, esses sistemas não oferecem nativamente um ponto de busca único que reúna a oferta de múltiplas locadoras concorrentes para o cliente final.

Do lado da demanda, agências de viagem *on-line* (OTAs) como Booking.com, Expedia e Kayak desempenham o papel inverso: agregam a oferta de diárias de múltiplas locadoras em um único ponto de busca para o cliente final, mas operam exclusivamente como uma camada de distribuição e reserva. A gestão da frota, das filiais, dos contratos e dos funcionários de cada locadora conectada a essas OTAs continua sendo feita por seus próprios sistemas internos — normalmente os mesmos sistemas fechados de tenant único descritos no parágrafo anterior — aos quais a OTA se conecta por meio de uma integração específica. *Marketplaces* de compar-

tilhamento de veículos entre pessoas físicas, como Turo e Getaround, resolvem um problema adjacente, mas distinto: conectam proprietários individuais de veículos a locatários, e não foram desenhados para representar a estrutura de uma locadora com múltiplas filiais, frota própria e funcionários.

Esse cenário evidencia uma lacuna entre três categorias de solução que, isoladamente, não atendem a uma locadora de pequeno ou médio porte que queira competir em um ponto de busca compartilhado: sistemas de gestão de tenant único, voltados às operações internas de uma única locadora e raramente preparados para modelar a logística entre filiais ou se conectar a múltiplos parceiros de distribuição; agregadores e distribuidores B2B, que compõem o *marketplace* de busca mas não gerenciam a operação das locadoras conectadas — a Rentalcars.com, operada pela Booking.com, reúne cerca de 800 locadoras em mais de 60 mil pontos de retirada, e a CarTrawler conecta mais de 1.700 fornecedores em mais de 50 mil localidades em 150 países, sempre como uma camada de distribuição sobre a operação interna de cada fornecedor [4]; e *marketplaces* de compartilhamento entre pessoas físicas, que não modelam a estrutura de filiais e frota de uma empresa de locação. A Tabela I resume essa comparação, incluindo critérios operacionais mais específicos identificados durante o desenvolvimento do CarRental Marketplace.

TABLE I
COMPARAÇÃO ENTRE CATEGORIAS DE PLATAFORMAS DE LOCAÇÃO DE VEÍCULOS

Critério	Gestão single-tenant	Agregadores/OTAs	Este trabalho
Gestão de frota e filiais própria	Sim	Não	Sim
Múltiplas locadoras em um só ponto de busca	Não	Sim	Sim
Sugestão automática de filial alternativa quando a categoria esgota	Raro	—	Sim
Taxa de devolução <i>one-way</i> modelada nativamente	Variável	—	Sim
Padronização de frota por código ACRIS	Variável	Geralmente exigida	Sim
Política de quilometragem (livre/limitada) com tarifa excedente	Variável	—	Sim
Planos tarifários com múltiplas moedas	Raro	Sim	Sim
Busca filtrável por nacionalidade e código promocional via API	Raro	Variável	Sim

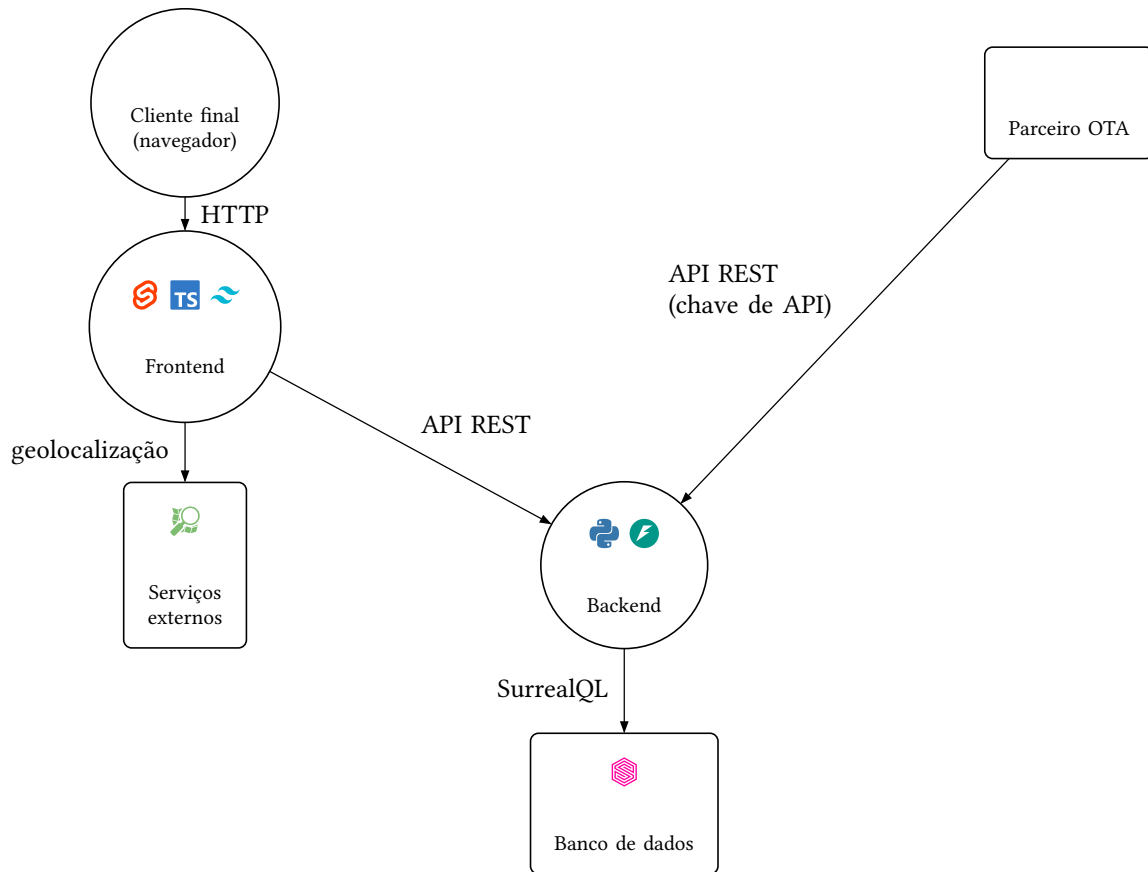


Fig. 1. Arquitetura do sistema e tecnologias utilizadas

O CarRental Marketplace se posiciona nessa lacuna: é, simultaneamente, o sistema de gestão da frota e das filiais de cada locadora cadastrada, o ponto de busca único para o cliente final entre as diferentes locadoras participantes, e uma API dedicada para que parceiros OTA externos comercializem essas diárias — com suporte nativo a planos tarifários com regras de elegibilidade e múltiplas moedas, taxas configuráveis por filial e por par de filiais (incluindo a taxa de devolução *one-way*), um controle de disponibilidade que evita o *overbooking* entre múltiplas filiais e, quando uma categoria esgota em uma filial, uma sugestão automática de filiais alternativas que possam suprir a reserva, já calculando o prazo e a taxa de transferência do veículo entre as unidades.

III. ARQUITETURA DO SISTEMA

O sistema adota uma arquitetura cliente-servidor desacoplada, dividida em três camadas principais: *frontend*, *backend* e banco de dados, conforme ilustrado na Figura 1.

O *frontend* foi desenvolvido com SvelteKit [5], um *framework full-stack* para a biblioteca Svelte que permite renderização tanto no servidor quanto no cliente, com tipagem estática garantida pelo TypeScript [6] e estilização utilitária pelo Tailwind CSS [7]. A camada de apresentação se comunica com o *backend* exclusivamente via chamadas HTTP a uma API REST, sem acesso direto ao banco de dados.

O *backend* foi desenvolvido em Python com o *framework* FastAPI [8], [9], escolhido por seu suporte nativo a *type hints*, geração automática de documentação OpenAPI e validação de dados de entrada e saída por meio de modelos Pydantic [10]. O *backend* é organizado em três camadas internas: *controllers* (rotas HTTP, agrupadas por domínio: autenticação, organização, frota, tarifação, reservas, administração e integração externa), *services* (regras de negócio) e o acesso a dados, sem uma camada intermediária de repositório ou ORM — os serviços montam e executam consultas SurrealQL diretamente sobre a conexão com o banco.

A *persistência* é feita no SurrealDB [11], um banco de dados multimodelo [12] que combina, em um único motor, tabelas de documentos e arestas de grafo nativas. Essa característica é explorada para modelar relações como o vínculo de propriedade entre usuário e empresa, o vínculo de trabalho entre usuário e filial, e as regras de devolução de veículos entre filiais, como relacionamentos de grafo de primeira classe (tabelas do tipo RELATION), em vez de tabelas associativas tradicionais.

A *segurança* é implementada em duas camadas de autenticação independentes: *tokens* JWT [13] para os usuários do sistema (clientes, funcionários de filial, gerentes de locadora e administradores), com controle de acesso baseado em papéis verificado por dependências de segurança no FastAPI; e chaves de API estáticas (cabeçalho X-API-Key) para parceiros OTA externos, validadas contra uma tabela dedicada de chaves ativas, sem necessidade de *login* de usuário.

IV. MODELAGEM DO SISTEMA

Modelo de domínio. O domínio da aplicação foi modelado por meio de um diagrama de classes [14] que captura as entidades centrais do negócio — usuário, empresa, filial, veículo, categoria de veículo, plano tarifário, taxa, adicional, proteção, cotação, reserva, contrato de locação, controle de disponibilidade e chave OTA — e o comportamento esperado de cada uma (por exemplo, a verificação se uma reserva pode transicionar para um novo status, ou o cálculo do valor de um adicional conforme seu tipo de cobrança).

Modelo de sistema. A arquitetura de implementação foi documentada por meio de um diagrama de classes que representa os *controllers* e *services* reais do *backend* e suas dependências, incluindo casos em que um único *controller* delega a múltiplos serviços (o *controller* de tarifação utiliza seis serviços distintos) e casos de reaproveitamento de função entre módulos (a função que resolve a empresa proprietária de uma filial, definida no serviço de reservas, é reutilizada pelos *controllers* público e de integração OTA).

Modelo lógico de dados. O banco de dados foi documentado por meio de um diagrama entidade-relacionamento lógico [15], derivado diretamente do esquema real do SurrealDB, contemplando 14 entidades de documento e 4 entidades associativas correspondentes às arestas de grafo nativas (vínculo de gestão, vínculo de trabalho, regra de devolução *one-way* e regra de transferência de frota entre filiais), com chaves primárias, chaves estrangeiras e cardinalidades de relacionamento — incluindo relacionamentos N:N implementados como *arrays* de identificadores de registro, em vez de tabelas de junção tradicionais.

Histórias de usuário e diagramas de sequência. O levantamento de requisitos resultou em 49 histórias de usuário [16], organizadas por perfil (cliente, gerente de locadora, operador de filial e administrador), cada uma correspondendo a uma tela real do sistema. Para cada história, foi elaborado um diagrama de sequência detalhando a interação entre o ator, a tela, o *backend* (camada “Sistema”), o *controller*, o serviço e o modelo de dados envolvidos, incluindo os principais fluxos alternativos de validação de regras de negócio.

V. FUNCIONALIDADES IMPLEMENTADAS

O sistema é organizado em cinco áreas funcionais principais:

- 1) **Organização.** Cadastro de locadoras (empresas), suas filiais e funcionários, com vínculos de papel (proprietário/administrador para usuário-empresa; gerente, atendente ou mecânico para usuário-filial, podendo um mesmo funcionário ser vinculado a mais de uma filial) e validação de CPF e data de nascimento no cadastro do cliente. Cada filial registra horário de funcionamento semanal, horários especiais (feriados ou eventos), tipo de localização (aeroporto, estação, centro urbano, hotel, *shopping*) e método de retirada (balcão, *shuttle*, *meet-and-greet*, a pé ou entrega no endereço do cliente).

- 2) **Frota.** Cadastro de categorias de veículo (com código ACRISS, capacidade de passageiros e bagagens, tipo de combustível, transmissão e ar-condicionado) e de veículos individuais, vinculados a uma categoria e a uma filial corrente. Cada categoria define uma política de quilometragem (livre ou limitada, com tarifa por quilômetro excedente).
- 3) **Tarifação.** Um motor de precificação que combina planos tarifários (com condições de elegibilidade por categoria, filial, idade do condutor, antecedência, nacionalidade e código promocional, podendo cada plano ser cotado em uma moeda diferente), taxas por filial (percentuais ou fixas, podendo ser sinalizadas como tributo e restritas a um intervalo de horário), adicionais opcionais e proteções/seguros com matriz de preço por categoria. O valor final de cada reserva é decomposto em um detalhamento item a item (tarifa-base, adicionais, seguros, taxas logísticas, taxas de condutor, tributos, descontos), o que dá rastreabilidade ao cálculo apresentado ao cliente.
- 4) **Reservas e disponibilidade.** Um fluxo de cotação (válida por um período limitado) seguido de confirmação de reserva, com verificação de disponibilidade por meio de um *ledger* de disponibilidade que contabiliza, por filial, categoria e data, o total da frota, o total reservado e o total em manutenção — evitando *overbooking* antes de cada confirmação. Quando uma categoria está indisponível na filial de retirada, o motor de cotação consulta as relações de logística cadastradas entre filiais e sugere automaticamente filiais alternativas que possuam a categoria disponível, já informando o prazo e a taxa de transferência do veículo. O ciclo de vida da reserva é controlado por uma máquina de estados (pendente, confirmada, ativa, concluída, cancelada ou não comparecimento) que valida as transições permitidas a cada status. Regras de devolução em filial diferente da retirada (*one-way*) também são suportadas, com taxa configurável por par de filiais.
- 5) **Administração e integração externa.** Um painel administrativo com estatísticas agregadas da plataforma e gestão de chaves de acesso para parceiros OTA, que consomem uma API dedicada (Seção VI) autenticada por chave de API.

O painel administrativo utiliza a biblioteca Chart.js [17] para a visualização gráfica de indicadores, e o cadastro de filiais utiliza a biblioteca Leaflet [18] para a seleção da localização geográfica em mapa.

VI. ESPECIFICAÇÃO DE INTERFACES DE PROGRAMAÇÃO (APIs)

O sistema expõe uma API REST interna [19], organizada por domínio (autenticação, organização, frota, tarifação, reservas, *dashboards*, administração e rotas públicas sem autenticação), consumida exclusivamente pelo próprio *frontend*. Separadamente, expõe uma API dedicada a parceiros OTA externos (*/ota/**), autenticada por chave de API em

vez de *token* de usuário, que reaproveita internamente os mesmos serviços de domínio utilizados pelas rotas públicas e do cliente final — ou seja, é uma fachada de autenticação alternativa sobre a mesma lógica de negócio, e não uma implementação paralela. Por essa API, um parceiro OTA pode listar as cidades com filiais disponíveis, buscar categorias disponíveis para um trecho e período filtrando por idade, nacionalidade e código promocional do condutor, solicitar uma cotação detalhada para uma categoria específica e criar, consultar ou cancelar reservas em nome de seus clientes — sempre restrito, pelas próprias credenciais da chave de API, à locadora à qual o parceiro está associado.

O *frontend*, por sua vez, consome três serviços públicos externos para funcionalidades de endereço e geolocalização, sem autenticação: o ViaCEP [20] para autocompletar o endereço de uma filial a partir do CEP; o serviço de geocodificação Nominatim, da OpenStreetMap Foundation [21], para converter um endereço em coordenadas; e o servidor de *tiles* da própria OpenStreetMap [21], para a renderização do mapa via Leaflet [18].

VII. RESULTADOS E DISCUSSÃO

A. Resultados Alcançados

O desenvolvimento resultou em uma plataforma funcional que cobre o fluxo completo de locação de veículos sob a perspectiva dos quatro perfis de usuário identificados (cliente, gerente de locadora, operador de filial e administrador), com suporte simultâneo à operação multi-filial de cada locadora, à composição de um *marketplace* entre locadoras concorrentes e à distribuição via API para parceiros OTA — as três frentes discutidas na Seção II. O motor de tarifação contempla planos com regras de elegibilidade, múltiplas moedas e taxas configuráveis por filial; o controle de disponibilidade evita o *overbooking* entre filiais e sugere automaticamente filiais alternativas quando uma categoria se esgota; e o ciclo de vida da reserva é integralmente controlado por uma máquina de estados.

Essa cobertura foi validada por meio dos 49 diagramas de sequência elaborados a partir do código real do *backend* e do *frontend* — e não de uma especificação idealizada — o que confere maior confiabilidade à documentação produzida: cada diagrama reflete o comportamento que o sistema efetivamente exibe para cada tela, e não apenas o comportamento originalmente previsto pelas histórias de usuário.

B. Limitações e Trabalhos Futuros

Esse mesmo processo de validação direto sobre o código permitiu identificar alguns pontos de melhoria, tratados como trabalhos futuros e detalhados na Seção VIII: (i) a ausência de implementação para o *checkout/check-in* de contratos de locação (*rental_agreement*) e para a persistência avulsa de cotações (*quote*), ambos presentes no esquema do banco de dados mas sem rota ou serviço correspondente; (ii) uma inconsistência de ciclo de vida entre as entidades de tarifação, em que planos tarifários, taxas e adicionais são desativados de forma lógica (*active = false*), enquanto proteções são removidas de forma definitiva; e (iii) o recálculo

independente do valor de adicionais em dois pontos distintos do *backend* (na pré-visualização da cotação e na persistência da reserva), o que representa um risco de divergência caso as regras de cálculo evoluam de forma assíncrona entre os dois pontos.

Nenhuma dessas limitações compromete o funcionamento do fluxo principal do sistema, validado na subseção anterior; elas indicam, antes, pontos de atenção para as próximas etapas de evolução do produto.

VIII. CONCLUSÃO

Este artigo apresentou o desenvolvimento do CarRental Marketplace, uma plataforma B2B2C de locação de veículos que conecta locadoras, filiais e clientes finais, com suporte a operação multi-filial e a integração com parceiros OTA — lacunas identificadas no mercado brasileiro de locação, caracterizado por um grande número de empresas de pequeno e médio porte sem acesso a esse tipo de solução. A plataforma foi construída sobre uma arquitetura cliente-servidor desacoplada (SvelteKit, FastAPI e SurrealDB) e documentada por meio de diagramas de domínio, de sistema, de modelo lógico de dados, de sequência e de especificação de APIs, derivados diretamente do código-fonte real do sistema.

Como trabalhos futuros, propõe-se a implementação dos fluxos de *checkout/check-in* de contrato de locação, a padronização do ciclo de vida das entidades de tarifação e a centralização do cálculo de precificação de adicionais em um único ponto do *backend*, além da condução de testes de validação com usuários reais do perfil de locadora e de filial.

REFERENCES

- [1] ABLA -- Associação Brasileira das Locadoras de Automóveis, «Setor de locação de veículos seguiu em crescimento durante o último ano». Acedido: 21 de junho de 2026. [Em linha]. Disponível em: <https://www.abla.com.br/noticia/setor-de-locacao-de-veiculos-seguiu-em-crescimento-durante-o-ultimo-ano-->
- [2] E. Calmon e J. Pestana, «Faturamento em locação de veículos cresce 16,6% e atinge R 61,7 bi em 2025». Acedido: 21 de junho de 2026. [Em linha]. Disponível em: <https://www.cnnbrasil.com.br/economia/mercado/locacao-de-veiculos-faturamento-2025-abla/>
- [3] J.-C. Rochet e J. Tirole, «Platform competition in two-sided markets», *Journal of the European Economic Association*, vol. 1, n.º 4, pp. 990–1029, 2003, doi: [10.1162/154247603322493212](https://doi.org/10.1162/154247603322493212).
- [4] AltexSoft, «Car Rental and Sharing APIs: Integrations with GDSs, OTAs, and Tech Providers». Acedido: 22 de junho de 2026. [Em linha]. Disponível em: <https://www.altexsoft.com/blog/car-rental-apis-integrations-with-gdss-otas-and-tech-providers/>
- [5] Svelte, «SvelteKit». Acedido: 21 de junho de 2026. [Em linha]. Disponível em: <https://svelte.dev/docs/kit/introduction>
- [6] Microsoft, «TypeScript». Acedido: 21 de junho de 2026. [Em linha]. Disponível em: <https://www.typescriptlang.org/>
- [7] Tailwind Labs, «Tailwind CSS». Acedido: 21 de junho de 2026. [Em linha]. Disponível em: <https://tailwindcss.com/>
- [8] FastAPI, «FastAPI». Acedido: 21 de junho de 2026. [Em linha]. Disponível em: <https://fastapi.tiangolo.com/>
- [9] Python Software Foundation, «Python». Acedido: 21 de junho de 2026. [Em linha]. Disponível em: <https://www.python.org/>
- [10] Pydantic, «Pydantic». Acedido: 21 de junho de 2026. [Em linha]. Disponível em: <https://docs.pydantic.dev/>
- [11] SurrealDB, «SurrealDB». Acedido: 21 de junho de 2026. [Em linha]. Disponível em: <https://surrealdb.com/docs/surrealdb>
- [12] F. Gessert, W. Wingerath, S. Friedrich, e N. Ritter, «NoSQL database systems: a survey and decision guidance», *Computer Science -- Research*

- and Development*, vol. 32, n.º 3–4, pp. 353–365, 2017, doi: [10.1007/s00450-016-0334-3](https://doi.org/10.1007/s00450-016-0334-3).
- [13] M. Jones, J. Bradley, e N. Sakimura, «JSON Web Token (JWT)», RFC7519, mai. 2015. Acedido: 21 de junho de 2026. [Em linha]. Disponível em: <https://www.rfc-editor.org/rfc/rfc7519.html>
 - [14] M. Fowler, *UML Distilled: A Brief Guide to the Standard Object Modeling Language*, 3.º ed. Addison-Wesley, 2004.
 - [15] P. P.-S. Chen, «The entity-relationship model: toward a unified view of data», *ACM Transactions on Database Systems*, vol. 1, n.º 1, pp. 9–36, mar. 1976, doi: [10.1145/320434.320440](https://doi.org/10.1145/320434.320440).
 - [16] M. Cohn, *User Stories Applied: For Agile Software Development*. Addison-Wesley, 2004.
 - [17] Chart.js, «Chart.js». Acedido: 21 de junho de 2026. [Em linha]. Disponível em: <https://www.chartjs.org/>
 - [18] Leaflet, «Leaflet». Acedido: 21 de junho de 2026. [Em linha]. Disponível em: <https://leafletjs.com/>
 - [19] R. T. Fielding, «Architectural styles and the design of network-based software architectures», 2000.
 - [20] ViaCEP, «ViaCEP». Acedido: 21 de junho de 2026. [Em linha]. Disponível em: <https://viacep.com.br/>
 - [21] OpenStreetMap Foundation, «OpenStreetMap». Acedido: 21 de junho de 2026. [Em linha]. Disponível em: <https://www.openstreetmap.org/>